Variational Latent Regularization for Amortized Chess Planning: A KL-Divergence Bottleneck Approach to Searchless Value Prediction

Harman Aujla, Vasilije Dragovic, Calissa Tyrrell, James Wu

Rensselaer Polytechnic Institute

April 25, 2026

Motivation

One key question: *Can we improve how transformers represent chess positions without adding search?*

Ruoss et. al 2024 showed that transformer-based chess agents can achieve strong play through **amortized planning**, learning to map board states directly to strong moves without explicit search.

- Strong performance, but important failure modes remain:
  - Overconfidence in objectively winning positions
  - Indecisiveness in critical tactical moments
  - Poor generalization to unfamiliar positions
- These suggest the model may play well while still learning **imperfect internal representations** of chess positions
- Our hypothesis: introducing a **structured latent bottleneck** forces the model to learn more robust and calibrated position representations
- Goal: improve decision quality *without adding explicit search*
- Why this matters beyond chess: **amortized planning** appears in robotics, control, and sequential decision-making more broadly

Background: Searchless Chess

ChessBench Setup

- Training data generated using **Stockfish** evaluations of large numbers of chess positions
- Each legal move is assigned an **action value** based on engine analysis
- A transformer is trained to predict these values directly from the board state

Inference Pipeline

$$\text{Board Position} \rightarrow \text{Tokenization} \rightarrow \text{Transformer} \rightarrow \text{Value Bin Prediction} \rightarrow \text{Argmax over Legal Moves}$$

- The model predicts a discrete value bin for each candidate move
- The highest-valued legal move is selected
- **No search at inference time** — decision making is entirely feedforward

Performance (Ruoss et al., 2024)

- 9M parameter model:
  - **88.9%** puzzle accuracy
  - **Tournament Elo 2025**
  - **Lichess Elo 2054 vs. Bots**
- 36M parameter model:
  - **94.5%** puzzle accuracy
  - **Tournament Elo 2259**
  - **Lichess Elo 2156 vs. Bots**
- 270M parameter model:
  - **95.4%** puzzle accuracy
  - **Tournament Elo 2299**
  - **Lichess Elo 2299 vs. Bots**
  - **Lichess Elo 2895 vs. Humans**

This shows strong chess performance is possible through **amortized planning** alone.

Architecture

- **Multi-Head Latent Attention** compresses full sequence $[B, T, H]$ into $L = 16$ learned tokens via cross-attention, then mean-pools to a single vector.
- **VAE Bottleneck** reparameterizes the pooled vector into $(\mu, \log \sigma^2)$, samples $z = \mu + \sigma \odot \epsilon$ and injects it back as an extra token.
- Our training objective adds **KL-divergence regularization** to the HL-Gauss loss: $\mathcal{L} = \mathcal{L}_{\text{HL_Gauss}} + \beta \cdot D_{\text{KL}}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, I))$

Architecture Diagram

Implementation

Core Idea

- Add a **variational latent bottleneck** to a searchless transformer

Pipeline

- Transformer backbone $\rightarrow$ latent compression $\rightarrow$ latent decoding

Key Components

- **Latent Attention:**
  - Learned queries compress sequence to a single vector
- **VAE Head:**
  - Predict (μ, σ), sample $z = \mu + \sigma \odot \epsilon$
- **Latent Injection:**
  - Append z as token $\rightarrow$ shallow decoder

Implementation Changes

- Forward pass returns $(\text{logits}, \mu, \sigma)$
- Add KL term to loss

Training

Setup

- ChessBench (Stockfish labels), single shard ($\sim$7M samples)
- $\sim$1M steps, batch size 1024

Objective

- $\mathcal{L} = \mathcal{L}_{\text{CE}} + \beta \cdot D_{\text{KL}}(q(z|x) \| \mathcal{N}(0, I))$
- Fixed $\beta = 10^{-3}$ (no annealing)

Training Details

- Adam + weight decay, mixed precision
- EMA weights for evaluation

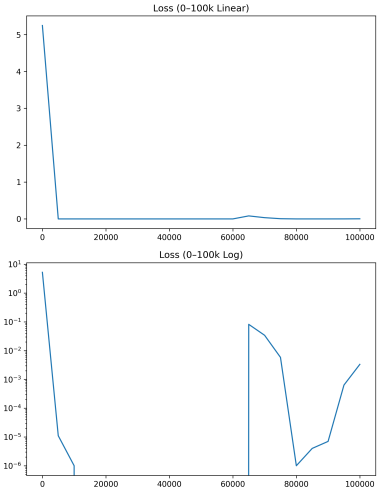
What We Track

- Loss, gradient norms, KL magnitude
- Latent stats: $\|\mu\|$, $\|\sigma\|$

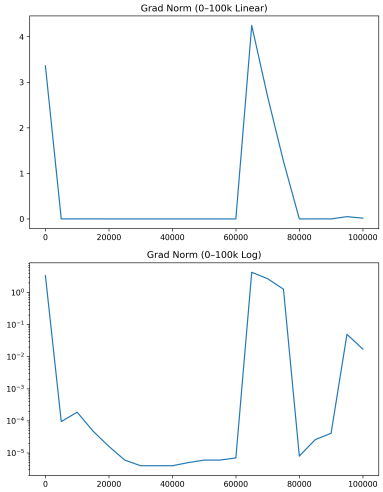
Result

- Early convergence $\rightarrow$ vanishing gradients
- **Posterior collapse**

Training Dynamics (Early)



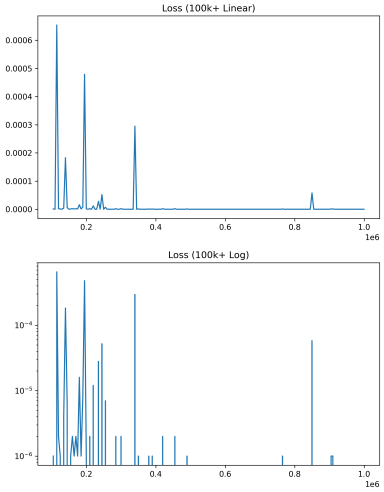
Loss



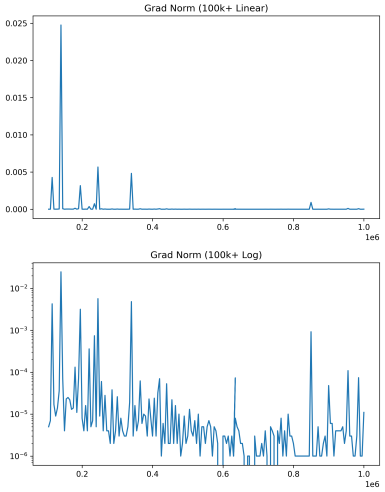
Gradient norm

Healthy optimization: loss decreases, gradients remain stable

Training Dynamics (Late)



Loss



Gradient norm

Collapse: near-zero loss, vanishing gradients, no learning

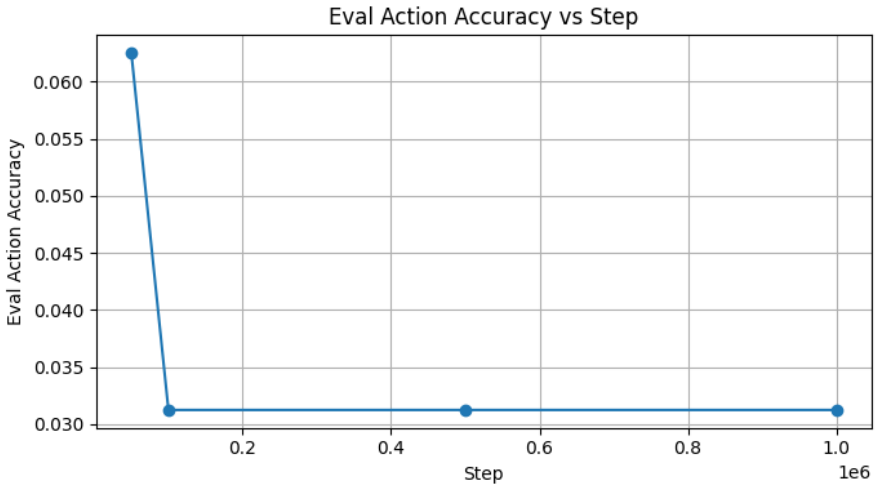
Evaluation Results

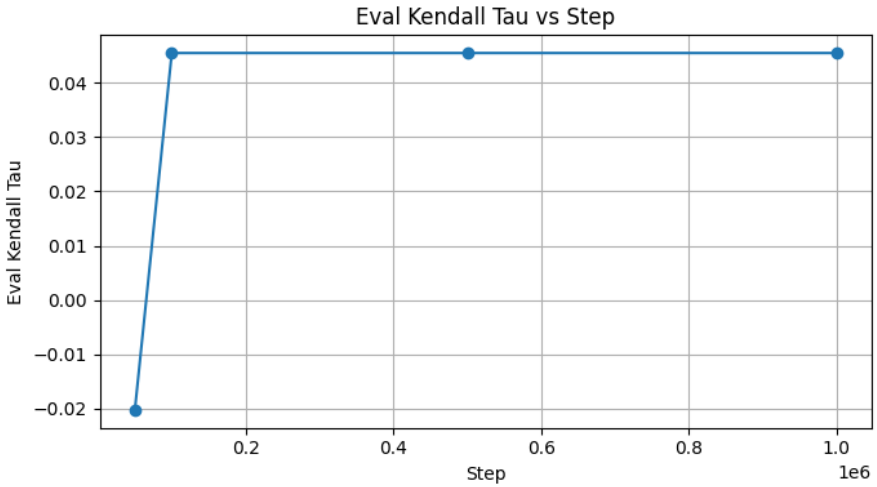
Evaluation Results at Different Steps

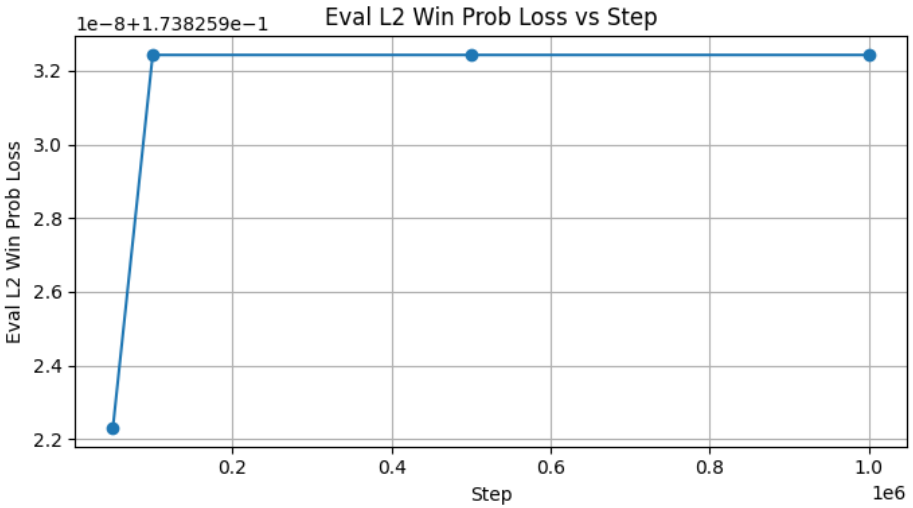
step	tree	eval_action_accuracy	eval_output_log_loss	eval_kendall_tau	eval_entropy	eval_l2_win_prob_loss
50000	params_ema	0.0625	30.972635	-0.020227146	1.3206342e-06	0.17382592
100000	params_ema	0.03125	506.09643	0.04546016	3.8371218e-25	0.17382593
500000	params_ema	0.03125	710.80741	0.04546016	0	0.17382593
1000000	params_ema	0.03125	750.33932	0.04546016	0	0.17382593

- There was a drop from 6.25% to 3.125% accuracy from 50k steps to 1 mil steps

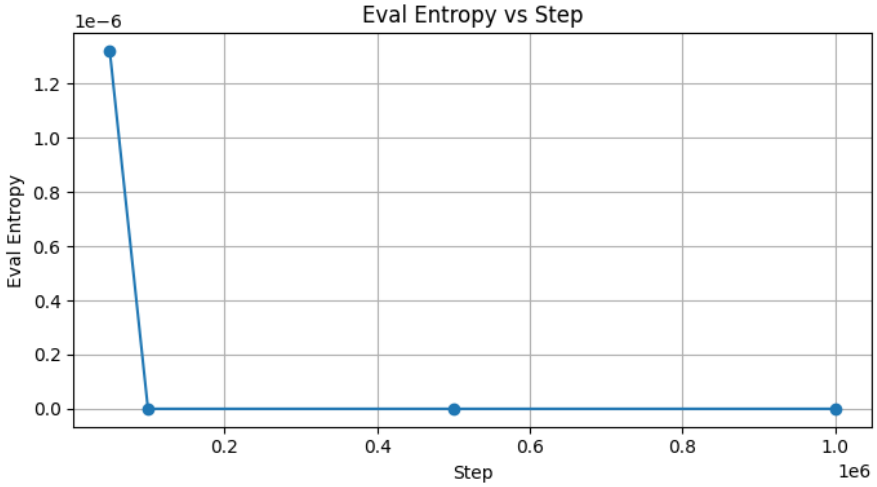
Evaluation Plots

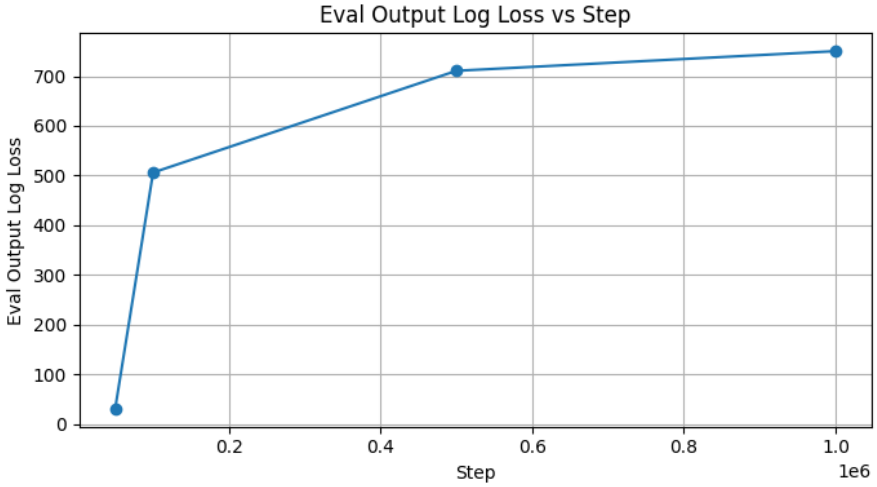






Diagnosing Posterior Collapse





- Around 100k steps, there is a posterior collapse, the model is very confident but wrong
- The log loss paradox occurs as a result

Why Did This Happen?

- **Data:** Single shard ($\sim$7M samples) was effectively memorized within $\sim$50K steps, providing no pressure to generalize
- **KL Weight:** $\beta = 10^{-3}$ applied from step 0 caused the KL term ($\sim$64 nats) to dominate reconstruction ($\sim$5 nats)
- **Decoder Capacity:** Transformer decoder was strong enough to ignore the latent variable entirely

Result: The model learns to bypass the latent $\rightarrow$ posterior collapse

What We Would Do Differently

- **KL Annealing:** Start with $\beta = 0$, gradually increase over $\sim$500K steps
- **Smaller β:** Reduce to $\beta = 10^{-5}$ to prevent early domination
- **More Data:** Train on 10–20 shards to introduce generalization pressure
- **Weaker Decoder:** Replace transformer decoder with a simple linear layer

Goal: Force the model to rely on the latent representation

Takeaways

- Implemented a **novel VAE bottleneck architecture** on top of a searchless chess transformer
- Provided a clear **empirical characterization of posterior collapse** in value distillation
- **Negative results are valuable:** we identified exactly why the approach failed
- The pipeline is complete and ready for **future experiments with targeted fixes**

Questions

Thank you!

Questions?